

SUP2I FOOD — ROADMAP NEXT

1. Geler le cahier des charges

Maintenant, on arrête d'ajouter des fonctionnalités au MVP.

Transformer toutes nos décisions en exigences précises :

- règles de commande ;
- statuts ;
- règles de paiement ;
- expiration ;
- créneaux ;
- quotas ;
- Food Pass ;
- permissions ;
- stock ;
- fidélité ;
- promotions ;
- notifications ;
- remboursements ;
- annulations ;
- exceptions.

Créer également un fichier :

DECISIONS.md

Exemple :

Commande non payée → jamais envoyée en cuisine.

Food Pass utilisé → impossible de réutiliser le même droit.

Responsable Cantine → ne peut pas modifier la photo.

Stock → source de vérité backend.

Résultat attendu

Un développeur doit pouvoir lire les règles sans devoir demander :

« Qu'est-ce qu'on fait dans ce cas ? »

2. Créer le workspace de développement

Mettre en place les projets réels :

sup2i-food/

└─ backend/

└─ mobile/

└─ web/

└─ docs/

└─ infrastructure/

└─ database/

Configurer :

- GitHub ;
- branches ;
- .gitignore ;
- conventions de commits ;
- Pull Requests ;
- Issues ;
- Projects/Kanban ;
- code review ;
- environnement .env.

Créer les colonnes :

BACKLOG

READY

IN PROGRESS

REVIEW

TEST

DONE

3. Transformer le projet en backlog technique

Créer les **Epics**.

EPIC-01 Auth

EPIC-02 Students

EPIC-03 Catalog

EPIC-04 Inventory

EPIC-05 Orders

EPIC-06 Payments

EPIC-07 QR

EPIC-08 POS

EPIC-09 Kitchen

EPIC-10 Slots & Queue

EPIC-11 Canteen

EPIC-12 Food Pass

EPIC-13 Notifications

EPIC-14 Promotions

EPIC-15 Loyalty

EPIC-16 Waste

EPIC-17 Administration

EPIC-18 Analytics

EPIC-19 Security

Puis transformer chaque Epic en petites tâches réalisables.

Exemple :

ORDER-001 Create Order entity

ORDER-002 Create OrderItem entity

ORDER-003 Create order endpoint

ORDER-004 Calculate order total

ORDER-005 Validate stock

ORDER-006 Generate order number

ORDER-007 Save status history

ORDER-008 Cancel order

ORDER-009 Expire unpaid order

4. Concevoir définitivement la base de données

C'est maintenant l'une des tâches les plus importantes.

Créer l'ERD complet.

Premières entités :

Organization

Campus

Location

User

Role

Permission

Student

Category

Product

ProductVariant

ProductOption

Ingredient

Recipe

RecipeItem

InventoryItem

InventoryMovement

StockReservation

Order

OrderItem

OrderStatusHistory

Payment

TimeSlot

SubscriptionPlan

Subscription

MealEntitlement

MealUsage

FoodPass

Promotion

Coupon

LoyaltyAccount

LoyaltyTransaction

Notification

WasteRecord

AuditLog

Définir pour chaque table :

- ID ;
- colonnes ;
- types ;
- relations ;
- contraintes ;
- indexes ;
- dates ;
- soft delete éventuel.

5. Définir les workflows comme machines à états

Ne pas laisser les statuts comme de simples textes.

Commande

DRAFT

→ CREATED

→ AWAITING_PAYMENT

→ PAID

→ QUEUED

→ PREPARING

→ READY

→ COLLECTED

→ COMPLETED

Branches :

CANCELLED

EXPIRED

REFUNDED

NO_SHOW

Paielement

PENDING

→ COMPLETED

ou

FAILED

REFUNDED

PARTIALLY_REFUNDED

Food Pass

ACTIVE

BLOCKED

LOST

REVOKED

EXPIRED

REPLACED

Abonnement

PENDING

ACTIVE

SUSPENDED

EXPIRED

CANCELLED

Le projet initial insiste déjà sur le principe **paielement avant préparation**.

6. Définir le contrat API avant l'intégration

Créer Swagger/OpenAPI.

Exemples :

POST /auth/login

GET /products

GET /products/{id}

POST /orders

GET /orders/{id}

GET /me/orders

POST /orders/{id}/pay

POST /orders/{id}/cancel

POST /scan/resolve

GET /canteen/menu

GET /me/subscription

GET /me/food-pass

POST /canteen/reservations

POST /food-pass/{token}/consume

Définir aussi le format standard des erreurs :

```
{  
  "code": "MEAL_ALREADY_USED",  
  "message": "Repas déjà distribué"  
}
```

Cela évitera que Mobile, Web et Backend inventent chacun leurs propres règles.

7. Définir les événements temps réel

Certaines parties doivent être instantanées :

- paiement effectué ;
- commande en préparation ;
- commande prête ;
- stock épuisé ;
- Food Pass utilisé ;
- commandes KDS.

Prévoir WebSocket ou SSE.

Exemple :

ORDER_PAID

ORDER_PREPARING

ORDER_READY

ORDER_COMPLETED

STOCK_CHANGED

8. Faire les maquettes UI/UX finales

Avant de développer tous les écrans, concevoir les parcours.

Mobile

- Login ;
- Home ;
- Catalogue ;
- Produit ;
- Panier ;
- Checkout ;
- Créneau ;
- QR ;
- commande active ;

- commandes prêtes ;
- historique ;
- Cantine ;
- Food Pass ;
- abonnement ;
- réservation ;
- favoris ;
- fidélité ;
- profil.

Web

- Dashboard ;
- produits ;
- catégories ;
- commandes ;
- stock ;
- étudiants ;
- Cantine ;
- Food Pass ;
- promotions ;
- gaspillage ;
- direction.

POS

- scanner ;
- panier ;
- paiement ;
- commande QR ;
- Food Pass ;
- Mode Rush.

KDS

- nouvelles ;
- préparation ;
- prêtes.

9. Installer l'infrastructure locale

Mettre en place :

Docker Compose

└─ PostgreSQL

└─ Backend

└─ services nécessaires

Configurer :

- migrations DB ;
- Flyway/Liquibase ;
- variables environnement ;
- profils DEV/TEST/PROD ;
- logs ;
- documentation API.

10. Développer le Backend Core

Commencer uniquement par les fondations.

Ordre conseillé :

Auth

↓

Users / Students

↓

Roles / Permissions

↓

Categories

↓

Products

↓

Pricing

↓

Orders

↓

Payments

↓

Stock

Ne pas commencer par fidélité ou IA.

11. Construire le premier flux de bout en bout

C'est votre **premier vrai objectif technique**.

Admin

Crée :

Coca

7 DH

Stock : 20

Backend

Stocke l'information.

Mobile

Affiche :

Coca

7 DH

Disponible

Résultat

WEB → BACKEND → DATABASE → MOBILE

Tant que ce flux n'est pas solide, ne passez pas aux fonctionnalités compliquées.

12. Construire la première commande complète

Deuxième vertical slice :

Mobile

↓

Produit

↓

Panier

↓

Créer commande

↓

Backend

↓

Dashboard

Résultat :

Commande #A001

AWAITING_PAYMENT

Ajouter :

- Order ;
 - OrderItem ;
 - prix figé au moment de la commande ;
 - historique ;
 - total ;
 - utilisateur ;
 - timestamp.
-

13. Ajouter le QR sécurisé

Ensuite :

Commande

↓

Token sécurisé

↓

QR

Le Mobile affiche le QR.

Le Web doit pouvoir résoudre :

SCAN

↓

ORDER A001

Prévoir :

- signature ;
- expiration ;
- statut ;
- révocation ;
- protection contre token falsifié.

14. Ajouter le paiement

Créer :

- paiement espèces ;
- paiement carte/TPE ;
- références ;
- historique ;
- remboursement.

Workflow :

AWAITING_PAYMENT

↓

PAY

↓

PAID

Une commande ne part toujours pas en cuisine avant PAID.

15. Développer le KDS Cuisine

Une commande PAID apparaît automatiquement.

A001

Tacos Poulet

Frites

Coca

[PRÉPARER]

Puis :

[PRÊTE]

Quand READY :

- Mobile actualisé ;
 - notification ;
 - file actualisée.
-

16. Développer le stock intelligent

Commencer par produit fini.

Ensuite ajouter :

- ingrédients ;
- unités ;
- recettes ;

- mouvements ;
- réservations ;
- disponibilité calculée.

Le projet prévoit explicitement les deux niveaux **stock produits + stock ingrédients**.

Exemple :

Tacos Poulet

└─ Poulet 150 g

└─ Fromage 40 g

└─ Tortilla 1

Si :

Poulet = 0

alors le produit devient automatiquement indisponible.

17. Sécuriser les stocks concurrents

Ajouter :

stockPhysical

stockReserved

stockAvailable

Tester :

deux personnes essaient d'acheter le dernier Coca.

Résultat attendu :

Utilisateur A → SUCCESS

Utilisateur B → OUT_OF_STOCK

Jamais stock négatif.

18. Développer les créneaux

Créer :

TimeSlot

startTime

endTime

capacity

reservedCapacity

status

Mobile :

12:30 — presque complet

12:45 — disponible

13:00 — disponible

Backend bloque automatiquement les créneaux pleins.

19. Ajouter la file virtuelle

Calculer :

- position ;
- nombre de commandes avant ;
- estimation.

Mobile :

A152

4 commandes avant vous

≈ 7 minutes

Ajouter l'écran :

COMMANDES PRÊTES

A145

A148

A152

20. Auditer maintenant le POS Cactus

Dès que tu fournis l'application complète :

Analyser :

- PHP ;
- framework ;
- structure ;
- routes ;
- DB ;
- produits ;
- ventes ;
- stock ;
- utilisateurs ;
- scanner ;
- imprimante ;
- tiroir ;
- TPE.

Puis choisir :

Cactus → API SUP2I FOOD

ou

Cactus modifié

ou

nouveau POS avec migration progressive

Ne rien casser avant cet audit.

21. Intégrer le scanner universel

Après audit POS :

SCAN

|

|— Barcode produit

|

→ ajouter panier

|

|— QR commande

|

→ ouvrir commande

|

|— Food Pass

|

→ ouvrir droits Cantine

|

|— Inconnu

→ erreur

22. Finaliser les fonctions caisse

Ajouter :

- panier ;
- quantités ;
- suppression ;
- remise autorisée ;
- paiement ;
- ticket ;
- tiroir ;
- session caissier ;
- clôture ;
- remboursements ;

- logs.

Puis :

Mode Rush

SCAN → PAYER → SUIVANT

23. Développer la Cantine

Après que Snack + POS fonctionnent correctement.

Créer :

- Menu ;
 - WeeklyMenu ;
 - SubscriptionPlan ;
 - Subscription ;
 - MealEntitlement ;
 - MealUsage ;
 - Reservation.
-

24. Développer le Food Pass

Créer :

- carte ;
- token ;
- statut ;
- étudiant ;
- date création ;
- date expiration ;
- historique.

Mobile :

SUP2I FOOD PASS

Abonnement :

ANNÉE

Quota :

75 repas

Aujourd'hui :

DISPONIBLE

25. Développer la distribution Cantine

Flux complet :

SCAN

↓

Food Pass valide ?

↓

Abonnement actif ?

↓

Jour autorisé ?

↓

Quota ?

↓

Déjà utilisé ?

↓

DISTRIBUER

Deux scans concurrents doivent produire :

1 SUCCESS

1 ALREADY_USED

26. Ajouter perte/remplacement Food Pass

Mobile :

Déclarer ma carte perdue

Backend :

ACTIVE → LOST/REVOKED

Admin :

Créer nouvelle carte

Conserver :

- abonnement ;
 - quota ;
 - historique.
-

27. Ajouter réservation Cantine

Étudiant :

Je mange demain

[OUI]

Dashboard :

Abonnés : 150

Réservations : 116

28. Ajouter notifications Push

Configurer Firebase.

Priorité :

1. commande prête ;
2. changement commande ;
3. Cantine ;
4. expiration abonnement ;
5. quota ;

6. promotions.

29. Ajouter favoris et recommander

Mobile :

- favoris produits ;
- anciennes commandes ;
- recommander.

Toujours revalider :

- prix ;
 - stock ;
 - options.
-

30. Développer Promotions

Créer moteur configurable :

- produit ;
- catégorie ;
- combo ;
- horaire ;
- dates ;
- utilisateur ;
- coupon ;
- quantité.

Exemple :

Menu Étudiant

Tacos + Frites + Coca

31. Développer Fidélité

Créer un ledger transactionnel.

+35 ORDER_A152

+20 BONUS

-100 FREE_DRINK

Ne jamais simplement modifier un total de points sans historique.

32. Ajouter Gaspillage

Créer :

WasteRecord

Avec :

- produit ;
 - ingrédient ;
 - quantité ;
 - coût ;
 - motif ;
 - date ;
 - utilisateur.
-

33. Ajouter Fournisseurs / Approvisionnement

Seulement après le stock principal.

- fournisseurs ;
 - prix achats ;
 - bons de commande ;
 - réception ;
 - lots ;
 - expirations ;
 - réapprovisionnement.
-

34. Construire le Dashboard Direction

Quand suffisamment de données existent.

Afficher :

- CA ;
- commandes ;
- panier moyen ;
- marge ;
- Mobile vs POS ;
- abonnements ;
- repas distribués ;
- stock ;
- ruptures ;
- gaspillage ;
- top produits ;
- heures de pointe ;
- satisfaction.

Puis le KPI stratégique :

Dépenses restauration captées sur campus.

35. Ajouter Audit complet

Tracer toutes les actions sensibles :

WHO

WHAT

WHEN

BEFORE

AFTER

TERMINAL

Exemples :

- changement prix ;

- stock ;
 - remboursement ;
 - carte remplacée ;
 - repas distribué manuellement ;
 - promotion ;
 - compte bloqué.
-

36. Hardening sécurité

Faire ensuite une passe dédiée :

- JWT ;
 - refresh token ;
 - RBAC ;
 - HTTPS ;
 - QR signé ;
 - rate limiting ;
 - validation serveur ;
 - protection injection ;
 - protection double requête ;
 - secrets ;
 - permissions ;
 - sauvegardes.
-

37. Mode réseau dégradé

Définir explicitement :

- ce que POS peut continuer à faire ;
- ce qui doit être bloqué ;
- cache local ;
- opérations à synchroniser ;

- conflits.

Food Pass doit être traité avec une attention particulière pour éviter les doubles distributions.

38. Tests automatisés

Créer :

Unitaires

- prix ;
- stock ;
- quotas ;
- promotions ;
- fidélité.

Intégration

- DB ;
- commandes ;
- paiement ;
- Food Pass.

E2E

Mobile → POS → Kitchen → Mobile

39. Tests de concurrence

Cas obligatoires :

Dernier produit

Dernier quota

Dernière place créneau

Même QR

Même paiement

Même Food Pass

40. Tests de charge

Tester des pics proches du comportement réel :

- 100 utilisateurs ;
- 200 ;
- 500.

Vérifier :

- temps réponse ;
- DB ;
- stock ;
- commandes ;
- WebSocket ;
- KDS.

41. Monitoring & sauvegardes

Mettre en place :

- logs ;
- erreurs ;
- disponibilité API ;
- métriques ;
- sauvegardes automatiques ;
- restauration testée ;
- alertes.

42. Créer un environnement DEMO

Charger des données réalistes :

500 étudiants

40 produits

20 ingrédients

100+ abonnés

300 commandes

promotions

gaspillage

points

stocks

Une démo avec une base vide donne une mauvaise impression.

43. Faire l'enquête étudiants

Cette partie reste également à réaliser.

Collecter :

- dépenses ;
- habitudes ;
- demande ;
- prix ;
- intérêt Snack ;
- Cantine ;
- précommande ;
- abonnements.

Cela servira à votre offre commerciale.

44. Construire le Business Case réel

À partir de l'enquête :

Nombre étudiants

× taux adoption

× panier moyen

× fréquence

=

potentiel

Créer :

- scénario prudent ;
- réaliste ;
- ambitieux.

45. Faire un pilote

Une fois le système stable :

- vrais produits ;
- vraie caisse ;
- vrais utilisateurs ;
- périmètre limité.

Mesurer :

- commandes ;
- ventes ;
- temps ;
- erreurs ;
- ruptures ;
- satisfaction ;
- gaspillage.

46. Corriger après pilote

Classer :

BLOCKER

CRITICAL

IMPORTANT

IMPROVEMENT

Corriger les Blockers avant présentation finale.

47. Préparer la démonstration officielle

Scénario Snack

Mobile

→ Commande

→ QR

→ POS

→ Paiement

→ KDS

→ Prête

→ Notification

→ Stock

→ Dashboard

Scénario Cantine

Food Pass

→ Distribution

→ Deuxième scan refusé

Scénario caisse classique

Barcode

→ Vente

→ Paiement

→ Ticket

→ Stock

48. Préparer le dossier commercial

Seulement après avoir des éléments concrets :

- problème ;
- enquête ;
- solution ;
- démonstration ;
- architecture ;
- sécurité ;
- résultats pilote ;
- business case ;
- potentiel ;
- conditions.

49. Préparer les trois offres

Licence

Vous gardez le logiciel.

Acquisition

SUP2I obtient des droits plus importants.

Partenariat

Master / financement / maintenance / licence / collaboration selon négociation.

50. Ce que vous devez faire immédiatement

Votre **NEXT immédiat** est donc exactement :

1. Geler PRD

↓

2. GitHub + backlog

↓

3. ERD complet

↓

4. Workflows / state machines

↓

5. API OpenAPI

↓

6. Maquettes UI

↓

7. Docker + PostgreSQL + projets

↓

8. Auth

↓

9. Catalogue

↓

10. Premier flux Web → Backend → Mobile

↓

11. Commande complète

Et votre **première Definition of Done** doit être :

Un produit créé depuis l'administration est sauvegardé par Spring Boot/PostgreSQL, apparaît réellement dans Flutter, peut être ajouté au panier et génère une commande visible dans le Dashboard.

Quand ce scénario fonctionne proprement, vous passez au **QR → paiement → POS → KDS**.

C'est maintenant le bon point de départ pour commencer réellement le développement, sans revenir constamment à la phase d'idéation.